National Higher School Of Artificial Intelligence
Faculty / Department of Computer Science

Internship Report

End-of-Year / Summer Internship

Analysis and Forecasting of Client Payment Behavior

Design and Development of a Streamlit Web Application for Cash vs. Online Encaissement Forecasting

Host organization: **Algérie Télécom**

Student:	**ACHOUR Djamel Eddine**
Field / Level:	AI and Data Science
Host company:	Algérie Télécom — Tiaret
Period:	September 6th – October 1st, 2026
Academic supervisor:	Ms. Boudri Khaira
Company supervisor:	Mr. El-Arbaoui Mohamed

Academic Year 2025–2026

Acknowledgements

First of all, I would like to thank the team at Algérie Télécom — Tiaret for accepting me as an intern and for giving me the time and the data I needed to do something useful, not just observation.

I am particularly grateful to my company supervisor, Mr. El-Arbaoui Mohamed, who let me move around freely during the first days and then trusted me when I proposed a project that was a bit outside what they had originally planned for me. He took the time to explain how billing really works behind the screens, and introduced me to the people in the commercial and technical services.

My thanks also go to the staff of the commercial agency, the billing (*facturation*) and collection (*recouvrement*) service, and the network intervention team, who patiently answered all my questions — even the naive ones — about invoices, unpaid bills, and how external entrepreneurs intervene for client connections.

I also thank Ms. Boudri Khaira, my internship supervisor at Algérie Télécom — Tiaret, for validating the subject and for the advice on structuring the machine learning part.

Abstract

This report describes a 25-days internship carried out at Algérie Télécom. The first three days were devoted to discovering the company: visits to several services, observation of the billing workflow, and understanding how Algérie Télécom works with external entrepreneurs (*sous-traitants*) who perform the physical network connection (*raccordement*) for new clients.

While this operational workflow was interesting, it was not directly exploitable as a development project with the data and tools available to me. During discussions with the billing service, I noticed something else: every day, the agency records thousands of payment lines — cash paid at the counter (*espèces*) and online payments — stored in large Excel files. Nobody was really using this history to anticipate the future.

I therefore proposed and developed a different project: a **payment behavior forecasting application**. The application learns from daily encaissement history (January to August 2026, about 627,000 transactions) and predicts future cash and online collections, in both amounts (DZD) and number of transactions. The system compares four regression models per target, keeps the best one, and exposes the forecasts through a Streamlit dashboard backed by a FastAPI backend, with the possibility to retrain the model when new files arrive.

Keywords: Algérie Télécom, encaissement, payment behavior, forecasting, machine learning, Streamlit, FastAPI, time series.

Contents

Acknowledgements **1**

Abstract **2**

1 General Introduction **7**

2 Host Organization and Discovery Phase **8**
2.1 Algérie Télécom in brief 8
2.2 The first three days: visits and observation 8
2.2.1 Day 1 — Welcome and commercial agency 8
2.2.2 Day 2 — Billing, collection, and "how an invoice lives" 9
2.2.3 Day 3 — Technical service and the entrepreneurs 9
2.3 Why the entrepreneur topic was not kept 10

3 Project Framing **11**
3.1 Problem statement 11
3.2 Objectives 11
3.3 Working method and timeline 12

4 Data Exploration and Forecasting Models **13**
4.1 Raw data description 13
4.2 Cleaning steps 13
4.3 Feature engineering 14
4.4 Models compared and results 15

5 Application Development: Streamlit Dashboard **17**
5.1 Architecture choice 17
5.2 Dashboard features 17
5.3 Retraining: append vs. replace 18
5.4 Demo and feedback 19

6 Conclusion **20**
6.1 Summary 20
6.2 Limits and future work 20
6.3 Personal assessment 21

List of Figures

4.1 Daily encaissement history (cash vs. online) — history view from exploratory analysis 14

5.1 Forecast line charts (history + forecast) and model update section 18

5.2 Dashboard demonstration view — KPIs, forecast table, and retraining cards 19

List of Tables

3.1 Indicative timeline of the internship 12

4.1 Features used for each target 15
4.2 Example of model comparison (illustrative values from notebook — replace with your exact MAE table) 16

Chapter 1: General Introduction

When I arrived at Algérie Télécom for my internship, I did not have a precise technical subject. The initial idea mentioned by the agency was vaguely "to discover the information system and help where needed."

What actually happened was much better. My supervisor told me: "For the first three days, you will not code. You will walk, observe, and ask questions." That is what I did, and that is what this report starts with.

This report is organized as follows:

- **Chapter 1** presents Algérie Télécom and what I saw during the discovery phase: the services I visited, how invoices are managed, and how external entrepreneurs handle the physical connection of clients.
- **Chapter 2** explains how I moved from that observation to my actual project: the problem I identified around payment history, and the objectives of the application I proposed.
- **Chapter 3** details the data work: exploration, cleaning, and the machine learning methodology for forecasting.
- **Chapter 4** describes the development of the Streamlit + FastAPI web application, its features, and how it is used.

I finish with a general conclusion, the limits encountered, and what I learned personally from this experience.

Chapter 2: Host Organization and Discovery Phase

Algérie Télécom in brief

Algérie Télécom is the historic public telecommunications operator in Algeria. It provides fixed telephony, ADSL / fibre (Idoom) internet access, and enterprise connectivity across all wilayas. Each wilaya has a regional direction with commercial agencies (*agences commerciales*), technical centers, and support services (billing, collection, finance, IT).

The agency where I did my internship combines two worlds that rarely talk to each other directly:

1. the **commercial / administrative side**: subscriptions, invoicing, cash collection at counters, online payment follow-up;
2. the **technical side**: network deployment, fault repair, and new client connections, often subcontracted to approved local entrepreneurs.

My internship took place mostly on the commercial side, with regular visits to the technical side.

The first three days: visits and observation

Day 1 — Welcome and commercial agency

On the first day I was welcomed by my supervisor, After a short tour of the building and introductions, I spent the day at the front office of the commercial agency.

What I saw: clients coming to pay bills in cash, others asking why their line was suspended, agents switching between two or three different software screens. The agent who mentored me that day showed me a typical client file: subscription contract, line number, outstanding invoices, payment receipts.

I was surprised by the volume of cash operations. Even though online payment exists, most people still come physically to pay.

Day 2 — Billing, collection, and "how an invoice lives"

The second day was spent with the billing (*facturation*) and collection service. This was the most instructive day for understanding money flow.

They explained the full cycle to me, in simple terms:

1. **Consumption / subscription:** each active line generates a monthly invoice (subscription + options + possible overruns).
2. **Issuing the bill:** invoices are generated centrally and made available at agencies and online.
3. **Payment:** the client pays either in cash at the counter (*espèces*), by cheque / transfer in some cases, or online (*paiement en ligne*). Each payment produces a line in the daily encaissement file.
4. **Follow-up:** unpaid invoices trigger reminders, then restriction, then suspension. The collection team monitors aging balances agency by agency.

They showed me the Excel exports they use every day. The files are huge, with one sheet per month (January, February, ...), and columns like payment method, amounts, cheque number, bank, date, department code. At that point I did not yet understand all the columns, but I noticed the files were never really analyzed — they were archived, printed sometimes, but not used for prediction.

Day 3 — Technical service and the entrepreneurs

The third day was completely different. I visited the technical / network service, which deals with the physical side: pulling cables, installing ONTs / modems, connecting new buildings.

Here I learned how Algérie Télécom works with **external entrepreneurs** (*entreprises sous-traitantes*):

- When a client requests a new connection in an area that needs extension work, the agency creates a work order.
- An approved local entrepreneur is assigned. He provides labor, sometimes small materials, and performs the *raccordement* (trenching, pole work, in-building wiring).
- Once the work is done, there is a reception (*réception des travaux*) by an Algérie Télécom technician, then the line is activated and billing starts.
- The entrepreneur is paid separately, on the basis of a contract / invoice per intervention — this circuit is completely separate from the client billing circuit.

The technicians explained this very clearly with examples, but I quickly realized two things: (1) this process is almost entirely paper-based and field-based, with very little digital data I could access, and (2) it had almost no connection with the billing Excel files I had seen the day before.

That evening I wrote in my notebook: "Entrepreneur workflow = interesting but not feasible as a coding project here. Billing data = messy but promising."

Why the entrepreneur topic was not kept

I discussed it frankly with my supervisor at the end of the third day. A project around "managing entrepreneurs and connections" would have required access to contracts, field planning tools, and GIS data that I did not have, plus coordination with people who are rarely in the office. It would have ended up as a theoretical mock-up with fake data.

The billing / encaissement data, on the other hand, was real, abundant, and directly available. The need was also real: the agency staff could tell me how much was collected *yesterday*, but had no tool to estimate *next week* or *next month*, especially with the slow but visible shift toward online payment.

We agreed together: I would focus my project on **payment behavior forecasting**. The discovery of the entrepreneur workflow would remain in my report as context (it explains how a client becomes billable in the first place), but the development work would be on encaissements.

That decision shaped the rest of my internship.

Chapter 3: Project Framing

Problem statement

Every day, the agency records thousands of payment operations. Cash payments dominate in volume and amount, but online payments are growing, particularly since June 2026 (as I later confirmed in the data). Two practical questions kept coming back in discussions:

- How much cash should we expect to collect in the coming days? This matters for cash management, staffing of counters, and treasury forecasts.
- Is the shift to online payment real and durable, or just a temporary peak? Should the agency encourage it further?

Currently, answers to these questions are based on experience and intuition ("August is usually like that") rather than on data. My project aimed to provide a simple, visual, data-driven answer.

Objectives

The objectives validated with my supervisor were:

1. **Understand the data:** explore the file `encaissement du 01-01-2026.xlsx` (January to August 2026), clean it, and extract reliable daily indicators.
2. **Compare forecasting models:** train several regression models to predict four targets — daily cash amount, online amount, cash transaction count, online transaction count — and select the best per target.
3. **Build a usable dashboard:** develop a Streamlit web application, connected to a FastAPI backend, showing history vs. forecast as line graphs over time, with comparison views and KPIs.
4. **Allow updates:** let a non-technical user upload a newer Excel file to retrain the model (append or replace mode) without touching code.

What was explicitly *out of scope*: real-time integration with the billing information system, user authentication, and automatic scheduling. The goal was a working internal prototype, not a production deployment.

Working method and timeline

I worked iteratively, roughly as follows:

Table 3.1: Indicative timeline of the internship

Week	Period	Main activity
1	Days 1–3	Service visits, billing and entrepreneur workflow discovery.
1	Days 4–5	Problem definition, first look at Excel files, project proposal.
2		Data exploration in notebook: cleaning, visualization, daily aggregation.
2		Feature engineering, model training and comparison, saving best model.
3		Development of FastAPI backend + Streamlit frontend, retraining endpoints.
3		Testing, demo to agency staff, report writing, feedback and fixes.

I kept a Jupyter notebook as a lab book (exploration + training), then moved the stable logic into a clean Python module used by both the backend and future retraining. I showed intermediate graphs to my supervisor every few days to make sure I was not going in the wrong direction.

Chapter 4: Data Exploration and Forecasting Models

Raw data description

The starting point was a single Excel file: `encaissement du 01-01-2026.xlsx`, with 8 sheets named `janvier` to `aout`. Each sheet has the same approximate layout: 12 columns, but the headers and order vary slightly from one month to another, and there are many empty rows and subtotal lines.

After inspection, I standardized the columns by position and renamed them as:

`method, cash_raw, timbre, total, cheque, virement, autres, no_cheque, code_banque, dir_banque, date, dept`

The total volume is around **627,000 rows** over the period **January 1 to August 31, 2026 (243 days)**. This is already a strong signal: the agency handles on average more than 2,500 payment lines per day.

Two critical facts emerged early and conditioned everything:

- **Cash rows** (method containing *Especes*) store the money in the `Montant Total` column.
- **Online rows** (method containing *paiement en ligne*, with many typos like *paiement e ligne*, *paiment en ligne*, *paiemnt en ligne*) store the money in the `Autres` column, with `Montant Total = 0`.

If you sum the wrong column, you get completely wrong results. I lost almost a day on this before understanding it by cross-checking a few daily totals with the billing team.

Cleaning steps

The cleaning, done in `notebook.ipynb` and later reproduced in `app/train.py`, was:

1. Keep only the first 12 columns to ignore comment columns on the right.

2. Normalize the payment method by lowercase substring match: `esp` → cash, `ligne|paiment|paiem` → online. All other / empty methods are dropped.

3. Parse dates with `dayfirst=True` and normalize to day. August sheets contain datetime strings with hours, handled by the same parser. Unparseable dates are dropped.

4. Convert `total` and `autres` to numeric (errors → 0).

5. Aggregate per day:

   - `cash_amount` = sum of `total` on cash rows,
   - `online_amount` = sum of `autres` on online rows,
   - `cash_count`, `online_count` = row counts per method.

6. Fill missing calendar days with 0 to obtain a continuous daily series (243 days).

Resulting totals: **about 551M DZD in cash (617,757 transactions)** vs. **about 16M DZD online (9,948 transactions)**. Cash is still overwhelming, but the online curve clearly rises from June to August — a point I highlighted in the dashboard.

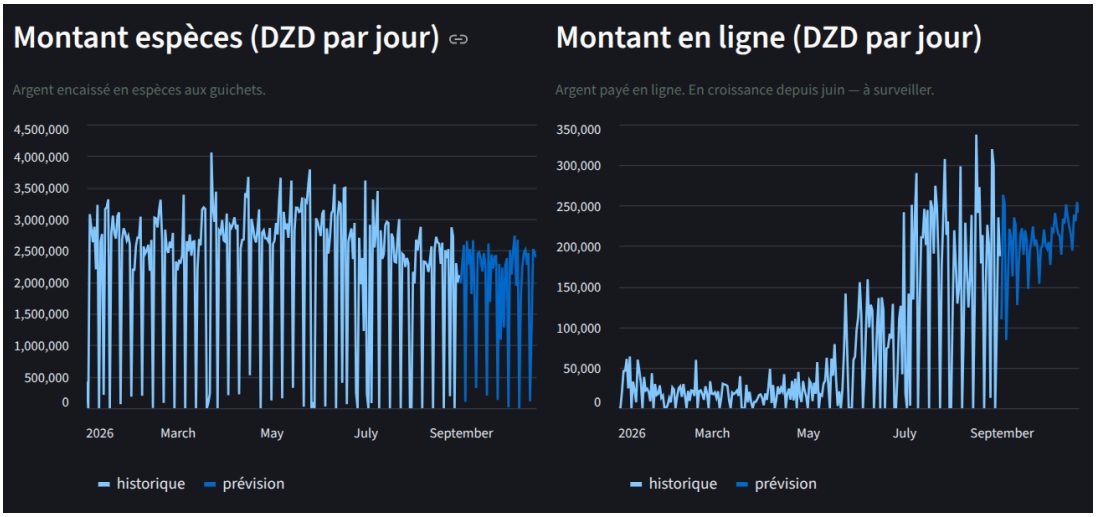


Figure 4.1: Daily encaissement history (cash vs. online) — history view from exploratory analysis

Feature engineering

Forecasting daily amounts directly from the date is not enough. For each of the 4 targets I built the same 8 simple features:

Table 4.1: Features used for each target

Feature	Meaning
`trend`	Days since start (captures growth)
`dow`	Day of week (0=Monday … 6=Sunday)
`month`	Month number
`dom`	Day of month
`weekend`	1 if Saturday/Sunday, else 0
`lag_1`	Value yesterday
`lag_7`	Value same weekday last week
`roll_7`	Mean of last 7 days

These features are deliberately simple and explainable to non-specialists: "yesterday, last week same day, recent average, plus calendar effects." More complex seasonality or external factors (holidays, salary days) were left for future work.

Models compared and results

For each target I compared 4 models: `LinearRegression`, `Ridge`, `RandomForest` (200 trees), and `GradientBoosting`, with a chronological 80/20 split (first 80% for training, last 20% for testing — no shuffling, to respect time order). Metrics: MAE, RMSE, R^2. The winner per target is the lowest MAE.

On this dataset the winners were:

- `cash_amount`: GradientBoosting,
- `online_amount`: LinearRegression,
- `cash_count`: GradientBoosting,
- `online_count`: LinearRegression.

This makes intuitive sense: cash series are noisy and non-linear (queues at month-end, weekends), so tree-based boosting helps; online series are smoother and more trend-driven, where a linear model generalizes better. Winners were refit on the full history and saved together in `best_model.pkl` as a bundle `{models, best_names, metrics, feature_cols, daily, targets}`.

Forecasting is **recursive**: to predict day $t + i$, I build features from calendar plus previous predictions (`lag_1`, `lag_7`, `roll_7`), predict, clip at 0, and append to history for the next step. Default horizon is 30 days (1–90 allowed).

Table 4.2: Example of model comparison (illustrative values from notebook — replace with your exact MAE table)

Target	Model	MAE	RMSE	R^2
cash_amount	GradientBoosting	**lowest**	—	—
cash_amount	RandomForest	2nd	—	—
online_amount	LinearRegression	**lowest**	—	—
online_count	LinearRegression	**lowest**	—	—

The main difficulty was not the algorithms themselves but the data quality: typos in method labels, mixed date formats, and the cash/online column trap described above. Once those were fixed, even simple models gave usable forecasts.

Chapter 5: Application Development: Streamlit Dashboard

Architecture choice

I chose a clean two-part architecture, as requested in the initial specifications:

- **Backend — FastAPI** (`app/backend.py`): loads `best_model.pkl` once at startup. Endpoints: `GET /` (health + winner names), `GET /history` (daily rows), `GET /predict?days=30` (forecasts), plus `POST /retrain/append` and `POST /retrain/replace` for updating the model with a new Excel file.
- **Frontend — Streamlit** (`app/app.py`): dashboard with KPIs, line charts, forecast table with CSV download, and two upload cards for retraining.
- **Shared logic** (`app/train.py`): `excel_to_daily()`, `merge_append()`, `train_bundle()` — the same code is used for initial training and for retraining, which avoids divergence.

Dependencies are listed in `app/requirements.txt`: `fastapi, uvicorn, streamlit, pandas, openpyxl, scikit-learn, matplotlib, requests` (plus `python-multipart` for file uploads).

To run locally (inside the `telecom` virtual environment):

```
./telecom/bin/uvicorn backend:app --reload --app-dir app
./telecom/bin/streamlit run app/app.py
# Backend on http://127.0.0.1:8000, frontend expects it there (BACKEND
   constant)
```

Listing 5.1: Running the application

Dashboard features

The Streamlit page is organized from top to bottom as a story a manager can read in one minute:

1. **Hero header + intro:** title, one-sentence explanation ("what clients paid yesterday, what the model expects tomorrow").
2. **Sidebar settings:** horizon slider (7–90 days, default 30), backend address reminder.
3. **KPI cards:** history days with date range, cash collected (M DZD + transactions), cash forecast and online forecast over the selected horizon.
4. **Forecast charts (line graphs over time):** cash history + forecast, online history + forecast, then two comparison graphs (forecast cash vs. online in amounts, and in transaction counts). Each chart shows history (past) and forecast (future) continuity.
5. **Forecast table:** expandable data table for the next N days with CSV download button.
6. **Update section:** two cards — *Append* (grow dataset, sums on overlapping dates, for newer months) and *Replace* (discard old history, train only on the new file, for corrected files). Each uploads the Excel file to the backend, retrains (1–2 min), and shows the result summary.
7. **About the model:** targets, current winners, features, training protocol, link to `TECHNICAL.md`.

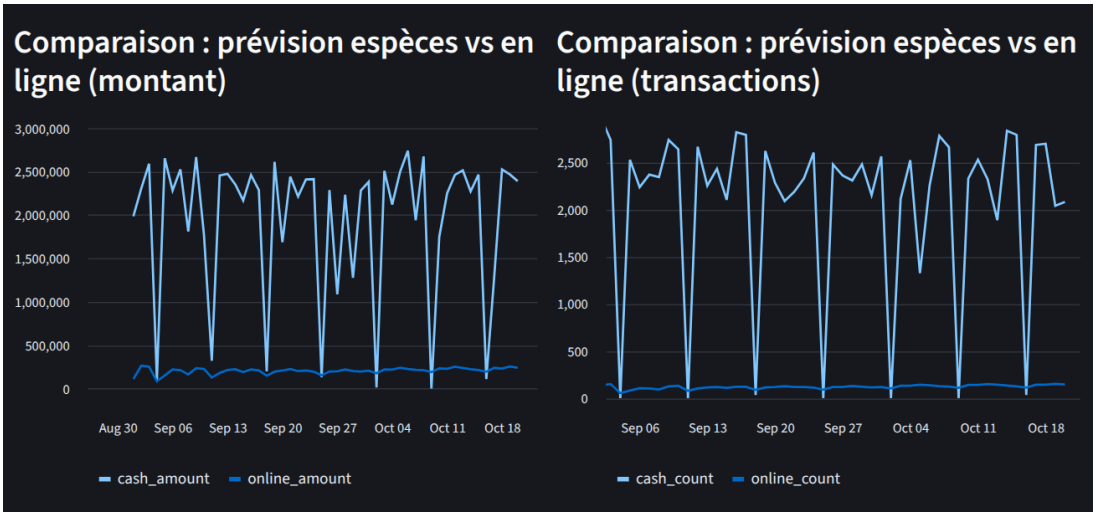


Figure 5.1: Forecast line charts (history + forecast) and model update section

Retraining: append vs. replace

This was the feature most appreciated during the demo, because agency staff immediately asked: "What happens next month when we have September data?"

- **Append mode** (`POST /retrain/append`): parses the new file into a daily table, outer-joins with existing history on date, **sums** values on overlapping days, fills gaps with 0, retrains all models, overwrites `best_model.pkl`, and returns `{mode, rows, date_from, date_to, best_models, metrics}`.
- **Replace mode** (`POST /retrain/replace`): discards old data and retrains only on the new file. Useful if the previous file had errors.

Both modes return the new date range and winners, displayed in the app. The user then refreshes the page to see forecasts from the enlarged dataset.

Demo and feedback

At the end of week 5 I presented the dashboard to my supervisor and two agents from the billing service. I used a 30-day horizon and showed the rising online curve. Their reactions were encouraging and concrete:

- They immediately understood the cash vs. online comparison graphs without explanation.
- They asked for a monthly total view ("give me September in one number"), which the KPI cards partially answer but could be improved.
- They pointed out that month-end and salary periods create peaks the model sometimes underestimates — a good lead for adding holiday / payday features later.

No production deployment was done during the internship (the app runs locally on demand.



Figure 5.2: Dashboard demonstration view — KPIs, forecast table, and retraining cards

Chapter 6: Conclusion

Summary

This internship gave me two complementary experiences. First, a field discovery of how Algérie Télécom really works: commercial reception, invoice lifecycle, and the role of external entrepreneurs in connecting clients — a world I knew nothing about before. Second, a complete data science mini-project, from raw Excel files to a working forecasting dashboard used and understood by non-technical staff.

Technically, I learned to deal with messy real data (typos, mixed date formats, misplaced amount columns), to build simple but honest time-series features, to compare models rigorously with a chronological split, and to wrap the result in a usable Streamlit + FastAPI application with retraining capability.

Limits and future work

The work has clear limits that I want to state honestly:

- Only 8 months of history (Jan–Aug 2026): too short to capture full yearly seasonality (Ramadan, summer, year-end).
- Simple features only: no holiday calendar, no salary-day indicator, no distinction between agencies.
- Recursive forecasting accumulates error over long horizons (beyond 30–45 days, treat numbers as trends, not precise amounts).
- No authentication, no database, no automated pipeline — retraining is manual via file upload.

Possible extensions: add holiday/payday features, test XGBoost / LightGBM or Prophet, aggregate by agency (`dept` column), add a monthly summary export (PDF), and eventually connect directly to the billing database instead of Excel exports.

Personal assessment

On a personal level, this internship taught me as much about communication as about code. Spending three days just listening before proposing anything felt slow at the time, but it is exactly what allowed me to propose a project people actually cared about. Presenting graphs to agents who had never seen machine learning and seeing them say "yes, that matches what we see at the counter" was more rewarding than any accuracy metric.

I leave Algérie Télécom with a better understanding of how a large public company operates, and with the confidence that I can take a vague need from observation to a deployed prototype.